Shared Names, Not Reflection: What Python Call Graphs Miss on Real Code, and Whether LLMs Can Find It

Sankalp Shankar
Dept. of AI & ML, Acharya Institute of Technology, Bangalore, India
sankalps.23.beai@acharya.ac.in

***Abstract*—Which calls do Python static call graphs miss on real code, and is the standard tool a stable instrument? Prior answers come from micro-benchmarks or count calls into built-in types that no analyser models. We run two static analysers, PyCG and a Pyright-based extractor, on repositories sampled from GitHub and score them, with three open-weight LLMs, against the calls each project's own test suite makes, excluding built-in calls and naming mismatches by construction. PyCG, the usual baseline, failed on 21% of 105 repositories (timeouts, crashes, out of memory), and on 46% of those it finished its raw output varied in size with the hash seed, by up to 65% of edges; across ten seeds the repository's own call graph changed on 20 of 76 repositories, moving PyCG's recall by up to 0.10. On 39 traced repositories, the union of both analysers has a median recall of 0.69; 69% of its misses go through method names shared by several functions, not reflection, and a reflection-based dynamism score predicts recall only weakly. Pyright finds more observed calls than PyCG in 33 of 39 repositories. LLM resolvers raise recall on hard items from 0.65 to 0.79–0.87 but lower precision from 0.83 to 0.60–0.70. A combiner of static and LLM answers, trained on other projects, raises F1 on unseen projects from 0.73 to 0.77. We release the pinned corpus, the pipelines and all measurements.**

***Index Terms*—call graphs, static analysis, dynamic analysis, Python, large language models, empirical study**

I. Introduction

A call graph records which functions call which. Tools for program comprehension, refactoring, dead-code detection and vulnerability reachability are built on one, and code-graph retrieval for LLM agents now uses it to decide which code a model sees [1]. Python makes call graphs hard to build statically: dispatch depends on runtime types, functions are passed around as values, and decorators, properties and registries hide calls behind syntax that does not look like a call.

PyCG [2] is the standard Python call-graph generator and the baseline in recent benchmarks [3] and in studies of LLMs as static analysers [4]. Its accuracy, like that of its successors [5], is mostly reported on hand-written micro-benchmarks, where every call has a known answer. Real repositories are larger, messier and have no answer key. For Java, executing programs to bound the recall of static call graphs is established practice [6], [7]; for Python, DyPyBench compared PyCG with dynamic call graphs on 50 projects and found only 49% of the dynamic edges in PyCG's output [3], but attributed 59% of the missing callees to methods of built-in types and about 12% to naming differences, rather than to analysis gaps. We exclude both by construction (Section III-B).

This paper asks what static analysers miss on real Python code once those artifacts are removed, whether the standard tool is a stable instrument, and whether LLMs recover the misses. Our numbers are not a correction of DyPyBench's: the corpus, the tracer and the edge definition all differ. We ask four questions:

- **RQ1 (reliability):** Does a widely used analyser produce a call graph at all, and the same one twice?
- **RQ2 (agreement):** How much do two independent static analysers agree on the same code?
- **RQ3 (recall):** Of the calls a project's tests actually make, how many do static analysers find, and where do they miss?
- **RQ4 (LLMs):** Can LLMs resolve the calls static analysis misses, and at what cost in false claims?

Our findings and contributions are:

1) **PyCG is not a fixed instrument.** The baseline of recent Python call-graph studies returns output of a different size per `PYTHONHASHSEED` on 46% of the repositories it finishes, by up to 65% of edges. Comparing exact edge sets over ten seeds, the repository's own call graph changes on 20 of 76 repositories, and PyCG's recall against observed calls moves by up to 0.10 (`term-image`: 0.34–0.43). It fails outright on 21% of the corpus, and on the timed-out repositories we probed its fixpoint diverges.
2) **Shared names, not reflection.** With calls into built-in types and naming mismatches excluded by construction, the union of two static analysers still misses a median of 31% of observed calls. Of the misses, 69% go through method names shared by several functions, and a dynamism score built on reflective constructs predicts recall only weakly ($\rho = -0.23$).
3) **A type checker is the better call resolver.** Pyright finds more observed calls than PyCG in 33 of 39 repositories, at similar precision (0.86 against 0.87).
4) **LLMs reverse the micro-benchmark verdict on recall, not on precision.** On real code with execution labels, LLMs find more observed calls than static anal-

TABLE I
THE CLOSEST PYTHON STUDIES AND THIS ONE

	DyPyBench [3]	Venkatesh et al. [4]	This paper
Code	50 real projects	micro-benchmark	105 real repos, 39 traced
Static tools	PyCG	PyCG	PyCG, Pyright
Seed determinism	not reported	not reported	3 seeds on all, 10 on 77
Built-in and naming artifacts	reported after measuring	not applicable	excluded by construction
Misses attributed to	built-in types, naming, `super()`	benchmark categories	shared method names, references
LLMs scored on execution	—	no	yes

ysis (0.79–0.87 against 0.65) but claim calls the tests never made. A static-plus-LLM combiner trained on other projects raises F1 on an unseen one from 0.73 to 0.77.

5) **A pinned corpus pipeline and all data:** discovery, dynamism scoring, automated build triage with recorded image digests and dependency locks, traces and every measurement.

II. RELATED WORK

Python call graphs. PyCG [2] computes an assignment graph over modules with a fixpoint and reports high precision and moderate recall on its micro-benchmark. Jarvis [5] adds flow sensitivity and reports higher precision, recall and speed than PyCG. Both evaluate primarily on micro-benchmarks and a handful of applications. We use PyCG because it remains the most common baseline, and we report its failure modes at scale.

Static versus dynamic call graphs. Sui et al. [6] and Helm et al. [7] execute Java programs to bound the recall of static call graphs and attribute misses to language features. DyPyBench [3] provides 50 executable Python projects and compares PyCG to DynaPyt-based dynamic call graphs.

LLMs as static analysers. Venkatesh et al. [4] evaluate 24 LLMs on call-graph and type-inference micro-benchmarks and find PyCG ahead of every LLM on call graphs. We revisit the question on real code with execution-derived labels.

What is new here. Table I sets this study against the two closest Python studies: a second static analyser on the same node set, a determinism check, known artifact classes removed before measuring, and LLMs scored against execution rather than benchmark labels.

III. STUDY DESIGN

A. Corpus

We searched GitHub for Python repositories with at least 50 stars, a push after August 2023, a `pyproject.toml` or `setup.py`, and a test suite. From the 98,416 matches we drew a seeded random sample (not ranked by stars), froze each repository's commit on the day of discovery, and scored its source with an AST-based dynamism score: weighted counts of dynamic constructs (`getattr` with a non-literal name, dynamic imports, `eval`/`exec`, registries, decorators) per thousand lines, with thresholds calibrated on a 14-repository reference set. Repositories were binned LOW, MED or HIGH and interleaved into a 558-repository queue, so that stopping early still leaves balanced strata.

An automated triage worked the first 240 repositories of the queue. Each was installed into a pinned `python:3.12` image (recorded by digest), falling back to 3.11 and 3.10 on interpreter or dependency errors, and its tests were collected with `pytest`. Every attempt records the resolved commit, the image digest and a hash of the frozen dependency list. 118 repositories built and collected at least one test; 64 built but collected none. After a licence gate, 105 entered static extraction.

B. Nodes and edges

Both analysers are mapped onto one node set, inventoried from the AST: modules, classes' bodies, functions, methods and lambdas, identified by file path and qualified name (line numbers are not part of identity). Comprehensions fold into their enclosing function; a method call resolves to the defining class in the method resolution order. An edge is *in-universe* when both caller and callee are callables of the repository itself; calls into builtins, the standard library and third-party packages are excluded, because no call-graph tool claims to model them and including them would inflate every miss rate.

C. Static analysers

PyCG 0.0.8 runs three times per repository, with `PYTHONHASHSEED` set to 0, 1 and 42. Each run uses its own container, an 8 GB memory cap and a 1800 s timeout, with the repository mounted read-only at the same fixed path, so the hash seed is the only input that changes between runs. The PyCG graph of a repository is the union of the seeds that finished.

Pyright 1.1.411 [8] has no batch call-graph mode. For every call site in the AST we ask `pyright-langserver` for the definition of the callee identifier and map the answer back to a node. Pyright runs without the repository's dependencies installed, which can only weaken it.

D. Runtime ground truth

For RQ3 and RQ4 we drew repositories from the 78 where both analysers finished, in two rounds whose rules were written down before either round was traced. The first took 20 (8 HIGH, 7 MED, 5 LOW): admitted on Python 3.12, at least 50 tests collected, a single top-level package, no heavy native stack and no external service. The second took every remaining repository with at least 20 tests, a single package and redistribution clearance, allowing native stacks, except two that need root or a live service (20 more). Of these, 3 could not be traced (Section IV-C). Where PyCG is seed-dependent, its graph here is the union of the three seeds of Section III-C. Each repository's own test suite runs under a tracer built

on Python 3.12's `sys.monitoring` [9]. It records every call between callables of the package, with the line it came from; calls from test code are attributed to a synthetic entry node, and a call that passes through library code (a callback handed to the standard library) is charged to the nearest package frame. We call these *observed calls*: they cover what one run of the tests executes and nothing else.

E. Call-resolution items

An *item* is a pair (caller, candidate) in which the candidate's name appears on a line of the caller that executed under the tests: as a call, as a bare reference (a property read, a function passed on) or as a class name (reaching its constructor). The label is whether the runtime call graph contains the edge caller→candidate: a property read counts through the getter it invokes, a class name through its `__init__`, and a bare reference only if a frame of this caller, or a library frame it called, made the call. Static tools are scored by the same edge test. For a uniquely named candidate on a line that executed, an unobserved claim is strong evidence of a false claim; for a shared name it is weaker, since another run could dispatch to it, so we report precision separately for the two. A resolver answers yes or no for each item: *name match* (every candidate is a call, as a text search would assume), PyCG, Pyright, their union, and the LLMs. Calls whose target never appears by name in the caller cannot be items; they are counted separately so they do not disappear from anyone's recall.

F. LLM resolvers and a combiner

We query gpt-oss-20b, gpt-oss-120b and qwen3.8-27b, open-weight models served by Groq, at temperature 0. Each prompt shows the caller's whole function body (its own lines, without nested definitions), the definitions of its candidates and the class hierarchy among them, and a JSON schema restricts the answer to candidate identifiers. The candidate list holds only names on lines that executed, the same filter that defines items; the models never see static analysis output. Every answer records the serving model, the prompt's SHA-256, temperature 0, seed 0 and reasoning effort *low*. Because of API rate limits, the LLMs answer a seeded sample of callers that contain at least one hard item (a name shared by several functions, a bare reference, or a pair the static analysers get wrong); every resolver is scored on the same sample. Each corpus repository contributes at most 25 sampled callers; `click` and `toolz`, traced earlier, contribute 140 and 60.

A logistic-regression combiner uses ten features: each static analyser's and each LLM's answer, how the name is written (call, reference, class name), whether it is shared by several functions, and whether the candidate is a method of the caller's own class. There is no preprocessing or feature selection, and the hyperparameters (L2 penalty 0.5, 3000 epochs, learning rate 0.3) were fixed while developing on `click` and `toolz`, before any corpus repository was traced. It is scored out-of-fold, once with folds grouped by caller function, so that no function's items are scored by a model trained on them, and once leaving out one whole repository at a time. Leave-one-repository-out is the primary result.

TABLE II
STATIC EXTRACTION OUTCOMES

Repositories attempted	105
both analysers finished	78
PyCG failed	22
timeout / crash / out of memory, every seed	15 / 4 / 1
failure mode varied by seed	2
package layout not detected	5
PyCG raw output size depends on seed (of 78)	36 (46%)
LOW / MED / HIGH dynamism	9/22 / 13/23 / 14/33
median / maximum edge-count spread	1.0% / 65%
repositories with spread $\geq$ 10%	7

IV. RESULTS

A. RQ1: Reliability of PyCG

Table II summarises static extraction. Pyright finished every repository whose package layout was detected. PyCG failed on 22 of 105 (21%): 15 hit the 1800 s timeout on every seed, 4 crashed on every seed (an `IndexError` while reading default arguments, or an `AttributeError`, both inside PyCG), 1 exhausted 8 GB, and on 2 the failure mode itself changed with the seed (a timeout on one seed, a crash or out-of-memory on another). When PyCG finishes it is fast (median 0.7 s per run, maximum 582 s), so the timeouts are not slow runs cut short. Size is part of the explanation, since failed repositories are larger (median 935 callable nodes against 181), but not all of it: `chevron`, with 16 callable nodes, timed out on every seed. The failure rate matches DyPyBench, where PyCG produced a call graph for 39 of 50 projects [3].

On 36 of the 78 repositories where PyCG finished (46%), its raw output differed in size between the three seeds. For the raw output this count-based check is a lower bound: two runs can return different edges in equal numbers. Because the counts of unique edges differ, output order or serialisation cannot explain it, and since repository path, image and tool version were identical across runs, the seed alone is responsible. The spread is usually small (median 1.0%) but has a long tail: 7 repositories vary by 10% or more, and on `movingpandas` the edge count varies by 65% between seeds. It occurs in every stratum. It had been noticed informally by tool builders [10]; we are not aware of a measurement of how often it occurs.

Mechanism. PyCG computes its graph with a fixpoint whose passes walk hash-ordered sets, and runs it with no iteration cap; set iteration order depends on `PYTHONHASHSEED`, which is random per process unless pinned. The seed therefore changes which fixpoint is reached, not only the order of the output. Comparing exact in-universe edge sets over ten seeds, the repository's own call graph changed on 20 of 76 repositories (minimum pairwise Jaccard 0.40, `python-lazy-object-proxy`). The count check misleads both ways: 16 of the 34 it flagged kept identical in-universe edges, and 2 it passed did change.

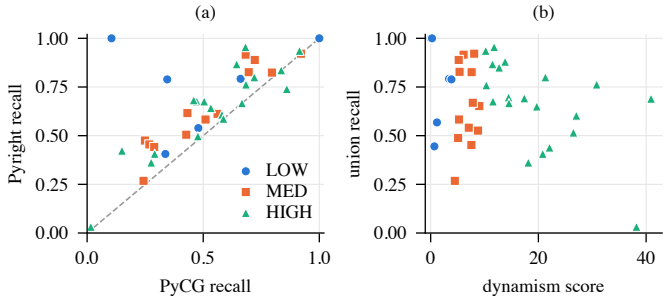


Fig. 1. Recall against observed calls, one point per repository. (a) Pyright against PyCG; points above the diagonal are repositories where Pyright finds more. (b) Union recall against the dynamism score used to stratify the corpus.

Timeouts are non-convergence. Replaying PyCG's own loop on `chevron` (16 callable nodes) with an iteration cap and a log per pass, its abstract state grew by 12 definitions on every pass, from 177 to 1420 over 100 passes, without converging or revisiting a state: each pass adds definitions under a freshly numbered lambda (`render.<lambda192>` by pass 50), so the convergence check can never succeed. On `muscriptor` the state exploded instead: the names it tracks grew from 555 to 18,387 in 4 passes as attributes accumulated on imported names, and the last pass alone took 22 minutes. Neither is a slow run that a larger budget would finish. Termination itself can depend on the seed: in the three-seed run the failure mode changed with the seed on 2 repositories, and in the ten-seed sweep 4 repositories finished on some seeds and ran past 30 minutes on others (e.g., `CorridorKey`).

Implication. PyCG's output is seed-dependent. Studies that use it should pin and report `PYTHONHASHSEED` or run several seeds and report the spread; a study that runs it unpinned inherits run-to-run variation in its results. They should also expect it to fail on roughly one repository in five.

B. RQ2: Agreement between static analysers

On the 78 repositories where both finished, the Jaccard similarity of the two analysers' in-universe edge sets has a median of 0.65 (interquartile range 0.56–0.80); 16 repositories fall below 0.5 and 19 reach 0.8 or more. Of the 33,368 edges found by either tool, 11% are found only by PyCG and 27% only by Pyright. Two analysers reading identical source disagree on roughly a third of the call graph. Agreement says nothing about which tool is right; that requires the runtime labels of RQ3.

C. RQ3: Static recall against runtime calls

We traced the test suites of the sampled repositories, plus `click` and `toolz` from the scorer's reference set. Of these, 3 could not be measured: `kolla-ansible`'s tests never import its own package, `wsgidav`'s suite produced no countable test results under the harness, and `recurrentgemma` keeps its tests beside the code (`foo_test.py`), which the harness cannot separate from package code. Across the remaining 39 repositories the tests exercised 10,882 distinct calls between functions and methods of the packages. Table III lists each repository.

How much is missing. The union of both static graphs contains a median of 0.69 of the observed calls per repository, from 0.03 to 1.00. Pyright alone reaches 0.68 and PyCG 0.53; Pyright finds more of the observed calls in 33 of 39 repositories (Fig. 1a), so in this corpus the type checker is the stronger call-graph source. Pooled over 33,020 call-resolution items (7,282 of them observed calls), the union has precision 0.85 and recall 0.70 (PyCG 0.87/0.51, Pyright 0.86/0.69). Plain name matching, which treats every name on an executed line as a call, has precision 0.22: most names in code that ran are not calls the tests made. Union precision is 0.88 on calls to uniquely named functions and 0.73 on shared names, where an unobserved claim may be a call that another run would make.

Where it misses. Of the 2,163 observed calls the union misses, 69% go to a method whose name is shared by several functions in the package, that is, dispatch over a class hierarchy or over unrelated classes with a common method name, and 38% are names written without call syntax, such as property reads and functions passed as values. The worst repository illustrates the first pattern: `ShivyC`, a C compiler, defines `make_il` on 29 syntax-tree classes and `make_asm` on 20, and the static union holds fewer than half of its observed calls. A further 3,659 observed calls (33% of all calls) reach a function never named in the calling function, so no resolver that reads the caller's code can find them: 38% are special methods invoked implicitly (operators, iteration, attribute hooks), and the rest are callbacks and functions reached through library code, 52% of them in one repository, `pysui`. Implicit calls also explain the lowest recall in Table III: `python-lazy-object-proxy` (0.03) is a proxy-object library whose tests exercise operator overloading, so almost every observed call is a special method that no call site names.

Dynamism predicts recall only weakly. The corpus was stratified by an AST score of reflective and dynamic constructs, but across repositories the score and union recall correlate only weakly (Spearman $\rho = -0.23$, Fig. 1b), and median recall is not ordered by stratum: 0.79 (n=6) for LOW, 0.65 (n=13) for MED and 0.69 (n=20) for HIGH. The constructs such scores count, reflection and dynamic imports, are not where static analysers lose most calls on real code; ordinary polymorphic dispatch is.

Coverage and stability of the labels. The tests executed a median of 67% of each package's statement lines (24% to 98%). Recall does not rise with coverage (Spearman $\rho = -0.33$ across repositories) or with the number of tests ($\rho = -0.23$); if anything, more thorough suites expose more of the hard calls. Tracing every suite a second time on 37 repositories, the observed-call sets of the two runs had a median Jaccard similarity of 1.00 (minimum 0.99).

Does the seed change recall? Across ten seeds, PyCG's recall against observed calls changed on 8 of 37 traced repositories, by up to 0.10 (`term-image`: 0.34–0.43; median 0.00), while the union's moved by at most 0.02: Pyright holds

TABLE III
STATIC RECALL AGAINST CALLS OBSERVED UNDER EACH TEST SUITE. S: STRATUM. PAIRS: DISTINCT CALLER→CALLEE CALLS OBSERVED. PYCG, PYRIGHT, UNION: SHARE OF THOSE CALLS IN EACH STATIC GRAPH.

Repository	S	Pairs	PyCG	Pyright	Union	Repository	S	Pairs	PyCG	Pyright	Union
ShivyC	L	1459	0.34	0.41	0.45	rejax	H	174	0.28	0.36	0.36
movingpandas	L	382	0.48	0.54	0.57	json-logging-python	H	86	0.29	0.41	0.41
rlm	L	142	0.34	0.79	0.79	tensortrade-ng	H	834	0.15	0.42	0.44
pr-agent	L	619	0.66	0.79	0.79	jsonapi-requests	H	153	0.48	0.50	0.52
TradingView-Screener	L	19	0.10	1.00	1.00	redbeat	H	63	0.59	0.59	0.60
throttled	L	13	1.00	1.00	1.00	pyrlp	H	120	0.53	0.64	0.65
pysui	M	1393	0.24	0.27	0.27	django-ses	H	3	0.67	0.67	0.67
sqllineage	M	761	0.29	0.44	0.45	aws-dynamodb-encryption	H	74	0.58	0.61	0.68
sec-edgar	M	123	0.27	0.46	0.49	apiflask	H	142	0.47	0.68	0.69
pykwalify	M	192	0.43	0.51	0.53	click	H	704	0.46	0.68	0.69
PyFunceble	M	1045	0.25	0.47	0.54	compressed-tensors	H	413	0.50	0.68	0.70
confluence-md-exporter	M	206	0.51	0.58	0.58	deepblast	H	29	0.69	0.76	0.76
term-image	M	336	0.43	0.62	0.65	marshmallow-sqlalchemy	H	38	0.68	0.76	0.76
toolz	M	121	0.56	0.61	0.67	scrapyrt	H	25	0.72	0.80	0.80
fastapi-injectable	M	46	0.70	0.83	0.83	xacro	H	140	0.84	0.84	0.85
all-repos	M	221	0.80	0.82	0.83	geocube	H	45	0.64	0.87	0.87
mcp-seq-thinking	M	36	0.72	0.89	0.89	autoray	H	50	0.86	0.74	0.88
bedrock-agentcore-sdk	M	356	0.68	0.92	0.92	neutronclient	H	47	0.92	0.94	0.94
creosote	M	63	0.92	0.92	0.92	puncover	H	22	0.68	0.95	0.95
lazy-object-proxy	H	187	0.02	0.03	0.03						
						Median over repositories			0.53	0.68	0.69

most of the edges PyCG gains or loses. The edges that change with the seed include observed calls on 8 repositories.

D. RQ4: LLMs as call resolvers

Table IV compares the LLMs with the static union on the sampled callers, which are enriched for hard items. Every LLM finds more of the observed calls than static analysis does: recall rises from 0.65 for the static union on the same items to 0.83 for gpt-oss-20b, 0.87 for gpt-oss-120b and 0.79 for qwen3.8-27b. Precision falls from 0.83 to between 0.60 and 0.70: the models claim calls the tests never made. The larger gpt-oss model has the highest recall and the lowest precision. On real code, then, the micro-benchmark finding that PyCG outperforms LLMs [4] holds for precision but reverses for recall. Resampling whole repositories, the 95% interval for recall is [0.78, 0.95] for gpt-oss-120b against [0.55, 0.74] for the static union, but the F1 intervals overlap ([0.64, 0.77] against [0.66, 0.78]), and an LLM has the higher F1 in only 15–18 of 39 repositories.

The LLMs complement static analysis rather than duplicate it: on observed calls the static union missed they recover 0.46–0.70, and on those it found they agree on 0.94–0.97. Table V shows where the gain comes from. Every resolver finds almost all calls to uniquely named functions. On shared method names static recall is 0.45 against 0.52–0.75 for the LLMs, and names written without call syntax, which static graphs almost never hold (recall at most 0.05), the LLMs recover at 0.74–0.86, but with precision of only 0.32–0.43.

The bottom of Table IV scores every resolver on the same items, leaving one whole repository out at a time. Adding every LLM claim to the static graph maximises recall at a large cost in precision. A majority vote of the three LLMs matches the static union on F1 (0.74 against 0.73). The learned combiner, trained on 5,573 items from 39 repositories, gives every source a positive weight and discounts names written without call syntax; on a repository it has never seen it reaches F1 0.77 (precision 0.81, recall 0.73, against 0.83 and 0.65 for the union). It beats the union on 20 of 39 repositories and loses on 9. With folds grouped by caller, which lets it train on other functions of the test project, it reaches 0.75, no better: what it learns is not specific to one project.

TABLE IV
CALL RESOLUTION ON THE LLM SAMPLE. TOP: EACH RESOLVER ON THE ITEMS IT ANSWERED. BOTTOM: THE ITEMS EVERY LLM ANSWERED, SCORED LEAVING ONE REPOSITORY OUT.

Resolver (items)	Precision	Recall	
gpt-oss-20b (5,611)	0.63	0.83	
gpt-oss-120b (5,616)	0.60	0.87	
qwen3.8-27b (5,578)	0.70	0.79	
	Precision	Recall	F1
PyCG	0.84	0.49	0.62
Pyright	0.83	0.64	0.73
PyCG ∪ Pyright	0.83	0.65	0.73
majority of the three LLMs	0.66	0.85	0.74
static ∪ any LLM	0.54	0.91	0.68
learned combiner	0.81	0.73	0.77

TABLE V
RECALL / PRECISION ON THE LLM SAMPLE BY HOW THE CANDIDATE IS NAMED IN THE CALLER.

Item kind	Items	Static	20b	120b	qwen
call, unique name	801	0.95 / 0.84	0.95 / 0.80	0.98 / 0.81	0.98 / 0.82
call, shared name	1798	0.45 / 0.74	0.67 / 0.60	0.75 / 0.57	0.52 / 0.71
class name	417	0.92 / 0.96	0.91 / 0.64	0.91 / 0.58	0.89 / 0.85
reference, unique name	1139	0.05 / 0.88	0.86 / 0.35	0.86 / 0.32	0.84 / 0.42
reference, shared name	1456	0.00 / 0.00	0.76 / 0.43	0.83 / 0.38	0.74 / 0.42

V. Discussion

Static call graphs are less settled than their benchmarks suggest. On micro-benchmarks PyCG is precise; on real repositories it fails on a fifth of the corpus and its output is seed-dependent on nearly half of the rest. The seed costs accuracy on some repositories (PyCG's recall moved by up to 0.10 across seeds), but the union with Pyright damps it to at most 0.02; the failures and the disagreement between analysers matter more. Tools that consume call graphs should treat each as one noisy view, and taking the union of two analysers is cheap.

LLMs are recall tools, not replacements. Static analysis is precise but blind to dispatch over shared names and to references; the LLMs see those but over-claim. Neither is enough alone, and combined through a filter learned on other projects they beat static analysis on unseen code. A tool that consumes call graphs can treat LLM answers as a second vote, not as a replacement.

Call graphs are not reference graphs. In a small pilot on repository-wide renames in `click`, every rename that failed even with runtime-derived callers in context (8 of 8) failed on a line that names the function without calling it: a dispatch-table entry, a callback registration, a monkey-patch. No call graph, static or dynamic, holds these lines; code assistants that retrieve context through call graphs need reference edges as well.

VI. Threats to Validity

Ground truth is workload-conditional. The tracer sees only what the test suites execute. Recall is measured against the calls the tests made, a subset of the calls the program can make, and precision only on lines that executed; a static edge that no test exercised is neither confirmed nor counted against the tool.

Environments. Static extraction (RQ1, RQ2) used the frozen environments of the triage run. The runtime traces for RQ3 and RQ4 were collected on a second x86-64 host where each repository was installed by the same recipe rather than from the frozen dependency list, so dependency versions may differ; the static graphs are computed from source and are unaffected. `click` and `toolz`, measured earlier, were extracted and traced on an arm64 development machine; both are pure-Python packages, so no compiled dependency enters their graphs.

Choice of analysers. PyCG's repository is archived and no longer developed, but PyCG is still the baseline in 2024–25 studies [3], [4], so its seed dependence bears on their numbers. We did not evaluate Jarvis [5], whose gains are reported on micro-benchmarks and a handful of applications; measuring it on the same traces is the obvious next step.

Tool versions and configuration. We measure one version of each analyser. Pyright ran without dependencies installed. PyCG's 30-minute budget is a choice; on the two timed-out repositories we probed, the fixpoint diverged, so a larger budget would not have helped there, but we did not probe all 15.

LLM sample and serving. The LLMs answered a sample of callers, enriched for hard items, so their scores are not directly comparable to the static scores on all items; we report every resolver on the same sample. Models served through an API may change; we record the serving model per answer.

Corpus. The corpus excludes repositories that cannot be installed from their root with `pip` or that collect no tests, which biases it against applications and monorepos. The runtime subset favours small, pure-Python packages whose test suites need no external services. The LOW stratum of the runtime subset is the thinnest, with 6 repositories.

VII. Conclusion

On real Python repositories, the standard static call-graph generator fails on a fifth of the corpus and its output depends on the hash seed on nearly half of the rest, and two static analysers disagree on a third of the edges. Measured against the calls tests actually make, the union of two static analysers finds a median of 0.69 of the calls, and its misses concentrate on shared method names rather than on the reflective constructs usually blamed. LLMs recover many of those calls at a cost in precision, and a learned combination of the two, trained on other projects, improves on static analysis alone. The pinned repository list, image digests, dependency-lock hashes, extraction and tracing code, and all results will be released upon publication, so that the numbers can be re-measured as tools change.

Acknowledgment

LLM queries were made through the co-authors' Groq API accounts, with Groq's permission.

References

[1] S. Ouyang *et al.*, "RepoGraph: Enhancing AI software engineering with repository-level code graph," arXiv:2410.14684, 2024.

[2] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "PyCG: Practical call graph generation in Python," in *Proc. 43rd IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2021, pp. 1646–1657.

[3] I. Bouzenia, B. P. Krishan, and M. Pradel, "DyPyBench: A benchmark of executable Python software," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 338–358, 2024.

[4] A. P. S. Venkatesh, R. Sunil, S. Sabu, A. M. Mir, S. Reis, and E. Bodden, "An empirical study of large language models for type and call graph analysis in Python and JavaScript," *Empirical Software Engineering*, vol. 30, no. 6, 2025.

[5] K. Huang, Y. Yan, B. Chen, Z. Tao, and X. Peng, "Scalable and precise application-centered call graph construction for Python," arXiv:2305.05949, 2023.

[6] L. Sui, J. Dietrich, A. Tahir, and G. Fourtounis, "On the recall of static call graph construction in practice," in *Proc. 42nd ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020, pp. 1049–1060.

[7] D. Helm, S. Keidel, A. Kampkötter, J. Düsing, T. Roth, B. Hermann, and M. Mezini, "Total recall? How good are static call graphs really?" in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024.

[8] Microsoft, "Pyright: Static type checker for Python," https://github.com/microsoft/pyright, version 1.1.411.

[9] M. Shannon, "PEP 669 – low impact monitoring for CPython," https://peps.python.org/pep-0669/, 2022.

[10] codellm-devkit, "codeanalyzer-python: pinning PYTHONHASHSEED for PyCG," https://github.com/codellm-devkit/codeanalyzer-python, commit ae2df74.